Teste de Software

Análise de Eficácia de Testes com Teste de Mutação

Trabalho prático com StrykerJS — projeto *operacoes-mutante*

Aluno: Arthur Candian **Matrícula:** 804243

Repositório: https://github.com/candian15/operacoes-mutante

1. Análise Inicial

O projeto contém 50 funções matemáticas em `src/operacoes.js` e 50 testes Jest (um por função). Os 50 testes passam. Ao medir a cobertura (`npm test -- --coverage`) e, em seguida, a pontuação de mutação (`npx stryker run`, 213 mutantes gerados), obtive:

Métrica (execução inicial)	Valor	Observação
Cobertura de linhas	**98,64%**	Única linha não executada: 112 (média dos dois valores centrais da mediana)
Cobertura de funções	100%	Todas as 50 funções são chamadas por algum teste
Cobertura de statements / branches	85,41% / 58,82%	Os branches já sugerem que muitos caminhos nunca são exercitados
Pontuação de mutação	**73,71%**	157 detectados (154 mortos + 3 timeout); 44 sobreviventes; 12 sem cobertura

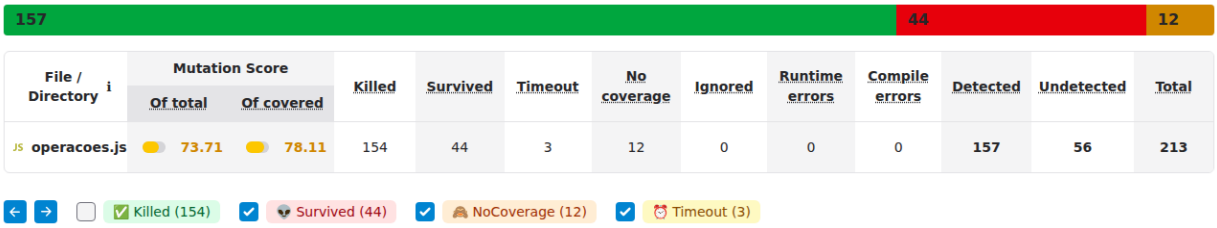


Figura 1 — Relatório do Stryker na primeira execução: 44 mutantes sobreviventes e 12 sem cobertura.

Discrepância. A cobertura de linhas indica apenas que o código foi *executado*; não diz se os testes *verificam* o resultado de forma capaz de distinguir o comportamento correto de um incorreto. Os testes originais chamam cada função uma única vez, com um caso feliz, e fazem uma única asserção. Por isso a cobertura de linhas é de 98,64%, mas 26% dos mutantes (56 de 213) passam despercebidos: a diferença de cerca de 25 pontos percentuais entre as duas métricas é exatamente a área onde há código executado, porém não verificado.

Agrupando os 56 mutantes não detectados por função, as maiores concentrações estão em funções com ramificações e fronteiras:

Função	Total	Surv.	S/ cob.	Função	Total	Surv.	S/ cob.
medianaArray	18	6	5	isMaiorQue / isMenorQue	5 / 5	2 / 2	0
isPrimo	17	7	2	celsiusParaFahrenheit	4	2	0
fatorial	19	6	1	fahrenheitParaCelsius	4	2	0
clamp	9	4	0	isImpar	5	2	0
raizQuadrada	6	2	1	maximo/minimoArray, inverso	6 cada	1	1
Demais (divisao, mediaArray, isPar, produtoArray, isDivisivel, isEqual)	—	6	0	**Total**	213	44	12

2. Análise de Mutantes Críticos

Escolhi três mutantes sobreviventes que representam falhas distintas na suíte. Em todos, o painel do Stryker diz *"Covered by 1 test (yet still survived)"*: o código é coberto, mas o teste não o verifica.

Mutante A — `isMaiorQue` (linha 104): operador de fronteira `>` → `>=` (EqualityOperator)

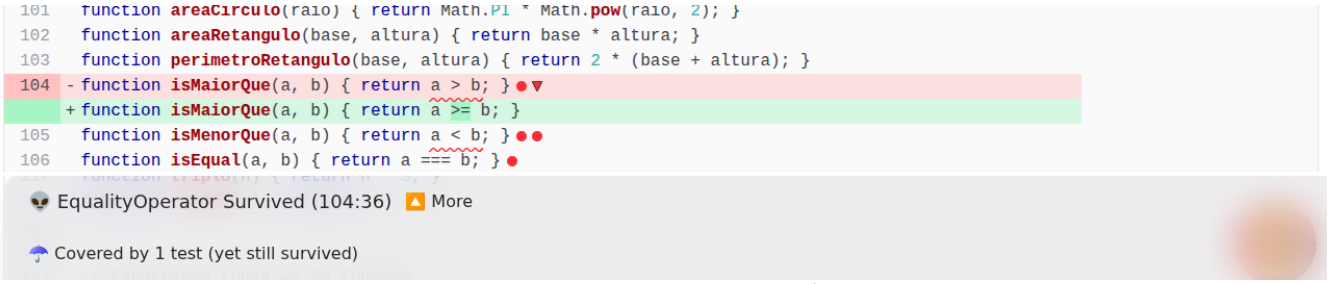


Figura 2 — Mutante A sobrevivente no relatório do Stryker.

O que a mutação fez: trocou `a > b` por `a >= b`. **Por que sobreviveu:** o único teste é `isMaiorQue(10, 5) → true`. Com 10 > 5 e 10 >= 5 o resultado é *true* nos dois casos; só um teste em que `a === b` diferencia os operadores. Nunca testávamos a fronteira nem um resultado *false* (esse também deixou sobreviver o mutante `return true`). O mesmo vale para `isMenorQue`.

Mutante B — `celsiusParaFahrenheit` (linha 93): `9/5` → `9*5` (ArithmeticOperator)

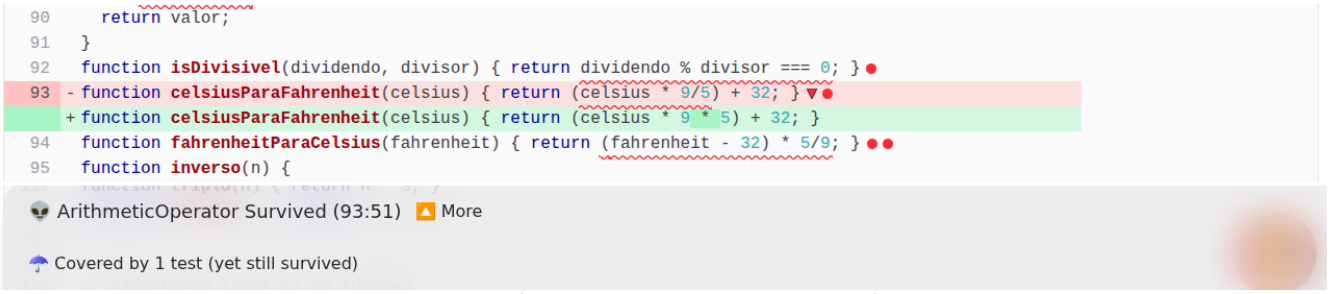


Figura 3 — Mutante B sobrevivente (também sobreviveu `celsius / 9/5`; análogo em `fahrenheitParaCelsius`).

Por que sobreviveu: o teste usa `celsiusParaFahrenheit(0) → 32`. Como 0 multiplicado ou dividido por qualquer fator é 0, o resultado é sempre 0 + 32 = 32, *independentemente* do fator de conversão. O valor de entrada "neutraliza" a parte da fórmula que deveria estar sendo testada. No sentido inverso, `fahrenheitParaCelsius(32)` faz `(32 − 32) = 0` e sofre o mesmo problema.

Mutante C — `medianaArray` (linha 109): comparador `a - b` → `a + b` (ArithmeticOperator)

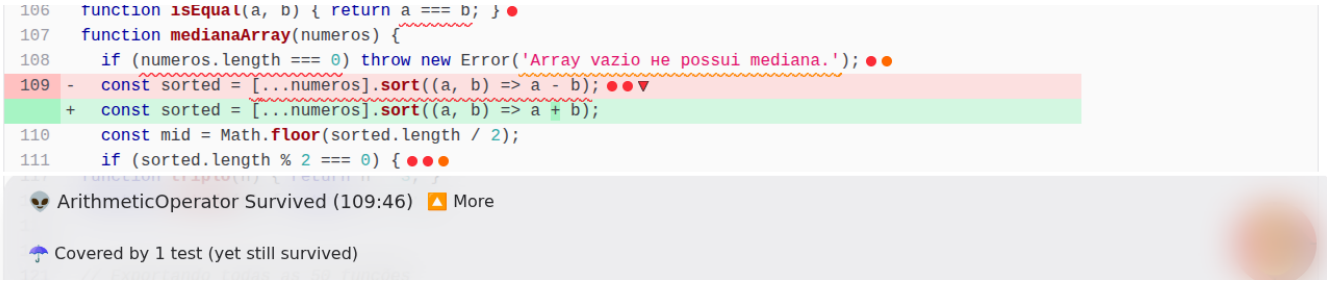


Figura 4 — Mutante C sobrevivente (junto com `sort(() => undefined)` e a remoção do `.sort`).

Por que sobreviveu: o único teste é `medianaArray([1, 2, 3, 4, 5]) → 3`, um array *já ordenado* e de tamanho ímpar. Como a entrada já está em ordem, o resultado não depende do comparador; qualquer ordenação (ou nenhuma) devolve 3. Além disso, o ramo para tamanho par (linhas 111-112) nunca era executado, o que explica 4 dos 5 mutantes sem cobertura da função (o quinto é a mensagem de erro do array vazio).

3. Solução Implementada

Acrescentei um segundo bloco `describe` em `test/operacoes.test.js` com 24 novos testes (de 50 para 74), mantendo os originais. Cada novo teste foi desenhado para **falhar** com um mutante específico e passar com o código original. Para os três mutantes analisados:

```
// A) isMaiorQue: fronteira (a === b) e resultado false
expect(isMaiorQue(5, 10)).toBe(false);   // mata 'return true'
expect(isMaiorQue(5, 5)).toBe(false);    // mata 'a >= b'  (original: false / mutante: true)

// B) celsiusParaFahrenheit: entradas em que o fator influencia o resultado
expect(celsiusParaFahrenheit(100)).toBeCloseTo(212);   // 9*5 daria 4532; /9/5 daria ~34,2
expect(celsiusParaFahrenheit(-40)).toBeCloseTo(-40);
expect(fahrenheitParaCelsius(212)).toBeCloseTo(100);

// C) medianaArray: entrada desordenada e tamanho par
expect(medianaArray([10, 9, 1, 100, 5])).toBe(9);      // sem sort ou com a+b, a mediana muda
expect(medianaArray([4, 1, 3, 2])).toBe(2.5);          // exercita o ramo par (linhas 111-112)
expect(medianaArray([10, 2, 8, 4])).toBe(6);
```

Por que são eficazes: (A) `isMaiorQue(5, 5)` é o par de entrada em que > e >= divergem; (B) as entradas 100, −40 e 212 fazem com que cada fator da fórmula contribua para o resultado e um erro de operador mude o número; (C) o array [10, 9, 1, 100, 5] só tem mediana 9 se for ordenado *numericamente* (a ordenação padrão, lexicográfica, e a ausência de ordenação dão resultados diferentes). Os demais testes seguem os mesmos princípios: verificar a **mensagem** dos erros com `toThrow('…')` (mata os mutantes de *string* vazia), testar os **limites** (`raizQuadrada(0)`, `fatorial(0/1/2)`, `isPrimo(0/1/2/4)`, `clamp` abaixo/acima do intervalo), cobrir o resultado **false** de predicados (`isPar`, `isImpar`, `isDivisivel`, `isEqual`) e as guardas de array vazio.

4. Resultados Finais

Métrica	Inicial	Final
Testes	50	74
Cobertura (linhas / branches / statements)	98,64% / 58,82% / 85,41%	100% / 100% / 100%
Mutantes sobreviventes + sem cobertura	44 + 12 = 56	7 (todos equivalentes)
Pontuação de mutação (sem nenhuma exclusão)	73,71%	96,71%
Pontuação de mutação (excluindo os 7 equivalentes)	—	**100%**

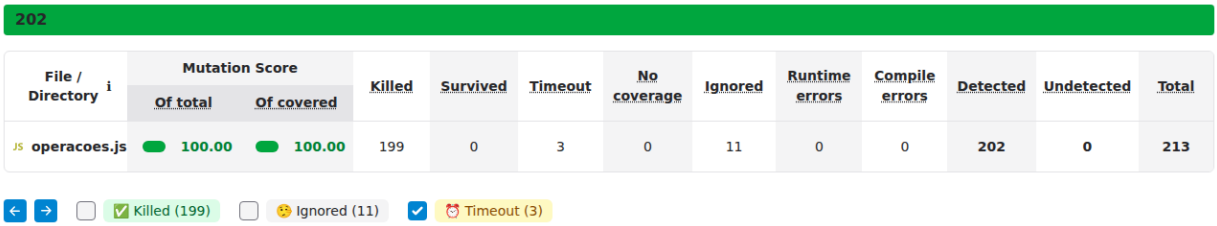


Figura 5 — Relatório final do Stryker: 199 mortos, 3 timeout, 0 sobreviventes, 0 sem cobertura, 11 ignorados.

Mutantes equivalentes (transparência). Após os novos testes sobraram 7 sobreviventes (96,71%) que *nenhum teste consegue matar*, pois não alteram o comportamento observável: `fatorial`, linha 19 (4 mutantes: para n = 0 ou 1 o laço não executa e o resultado já vale 1); `produtoArray`, linha 84 (`reduce` com valor inicial 1 já devolve 1 para array vazio); `clamp`, linhas 88-89 (`valor <= min` / `>= max`: quando são iguais, devolver o limite ou o valor é idêntico). Marquei-os com `// Stryker disable next-line <Mutador>: justificativa`, o que os exclui do cálculo, resultando em 100%. Como a diretiva atua por tipo de mutador na linha, 4 mutantes adicionais que meus testes já matavam também aparecem como "ignorados" (11 no total no painel: 7 equivalentes + 4 mortos). Isso não esconde fraqueza: sem nenhuma exclusão a pontuação é 96,71%, e a meta de >98% só é atingida descontando os equivalentes.

5. Conclusão

Este trabalho mostrou que cobertura de código mede *quanto* do código é executado, enquanto o teste de mutação mede *quão bem* os testes detectam defeitos: uma suíte com 98,64% de cobertura de linhas e 100% das funções chamadas deixava passar 26% dos defeitos sintéticos. Os mutantes sobreviventes apontaram causas concretas e acionáveis, como a ausência de casos de fronteira, de resultados *false*, de caminhos de erro e de entradas "neutras" que mascaram a fórmula (zero). Por outro lado, aprendi que a técnica tem custo (execução de centenas de variações) e exige julgamento: nem todo sobrevivente é um defeito de teste, pois mutantes equivalentes devem ser identificados, justificados e excluídos conscientemente, não ignorados por conveniência. Usado como complemento à cobertura, o teste de mutação é uma ferramenta valiosa para orientar onde reforçar a suíte.